Control de versions, documentació i integració continua

github Pages, colaborar en projectes i bones pràctiques

Desplegament d'aplicacions web
2 DAW

Github pages

https://pages.github.com/

- Ens permet crear o publicar una pàgina web per al nostre projecte.
- Els continguts de la pàgina web poden estar en un README.md o en fitxers .html
- Per utilitzar-lo, el més recomanat i habitual és disposar d'una branca gh-pages (encara que pot no ser així) on només hi haja el contingut associat a la documentació o pàgina web.

Github pages

https://pages.github.com/

Principals passos per disposar de github Pages al nostre repositori:

- Crear el repositori
- Crear el fitxer readme o l'estructura de fitxers html del nostre lloc i pujar-los al repositori remot. Podem fer-ho en una branca específica (gh-pages) o en qualsevol altra.

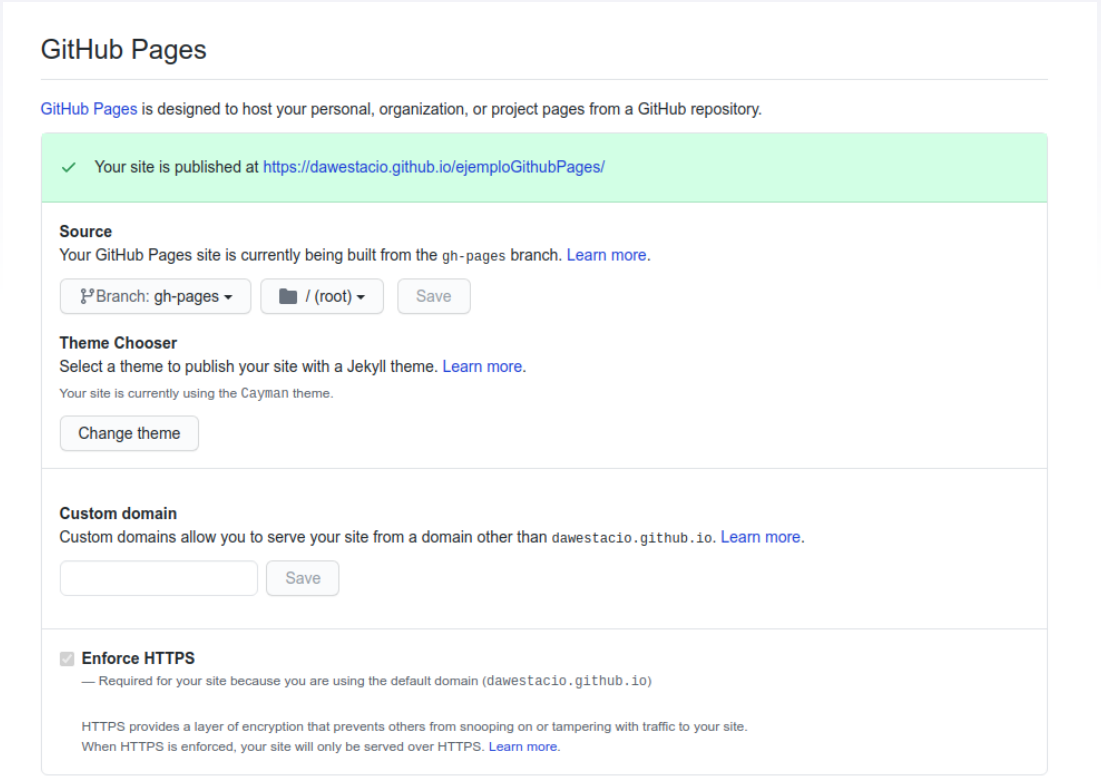
Github pages

https://pages.github.com/

- Habilitar github Pages al nostre repositori. Per això, una vegada dins del repositori, accedir al menú de Settings i marcar:
  - La branca associada on buscar el contingut de la documentació
  - Seleccioneu un tema per millorar l'aspecte final. Aquesta opció no és obligatòria i només serà efectiva si la nostra documentació està en un README.md

Github pages

https://pages.github.com/

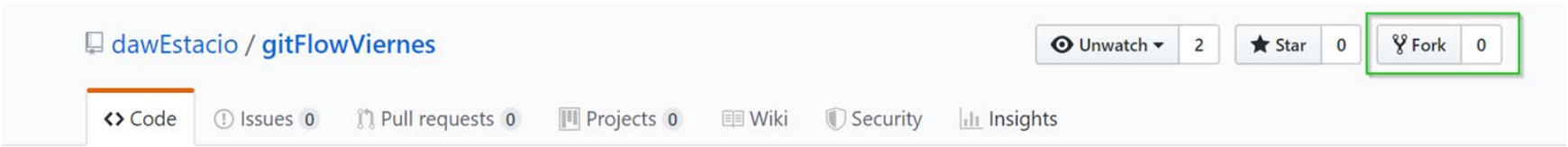


Col·laborar en projectes

Fork d'un projecte

Fer fork a un repositori significa copiar-lo. Copiar un repositori ens permet experimentar lliurement sense afectar el projecte original.

Quan es fa Fork d'un repositori es fan modificacions sobre aquest repositori per manipular la versió original sense causar problemes.



Col·laborar en projectes

Fork d'un projecte

Una vegada realitzat el fork d'un projecte, disposem d'un repositori com qualsevol altre. Això ens permet, entre altres coses, poder clonar-lo per treballar de forma normal amb ell.

Una pràctica habitual que sol realitzar-se sobre el repositori copiat és indicar a git que disposem d'una ubicació remota. Per això, farem:

$git remote add upstream direccion_repositorio_original

Li estem indicantque afegiu la següent **ubicació remota** i li diem **upstream**. Upstream és un terme en anglès que es pot interpretar com a "principal" o "producció", però si desitgem li podem canviar de nom, ja que el nom **upstream** només es fa servir per estàndard i no és que siga una cosa obligatòria

Col·laborar en projectes

Fork d'un projecte

Una vegada creada la branca upstream, podem actualitzar-nos amb els seus canvis i fer el merge a les nostres branques de treball per poder disposar dels darrers canvis del projecte original.

```
$git fetch upstream
```

El següent pas consistirà a fer un merge per fusionar els nous canvis del projecte original amb el nostre repositori copiat.

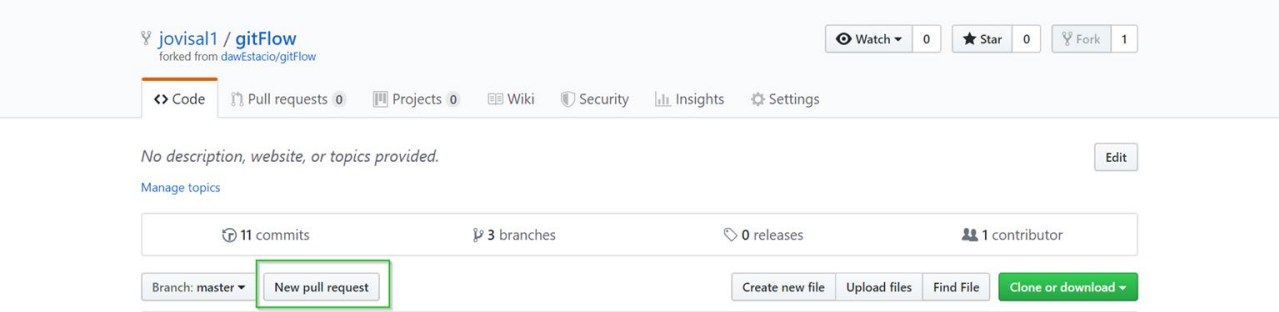
```
$git checkout master
$git merge upstream
```

Col·laborar en projectes

Pull request

Una vegada hem creat el nostre fork i hem afegit els canvis oportuns, el següent pas és proposar el propietari del repositori original que els incloga al seu projecte. Per això existeixen a GitHub els pull request.

Per obrir un pull request anem a la web del repositori original a partir del qual vam fer el nostre fork, i fem click a "**New pull request**".



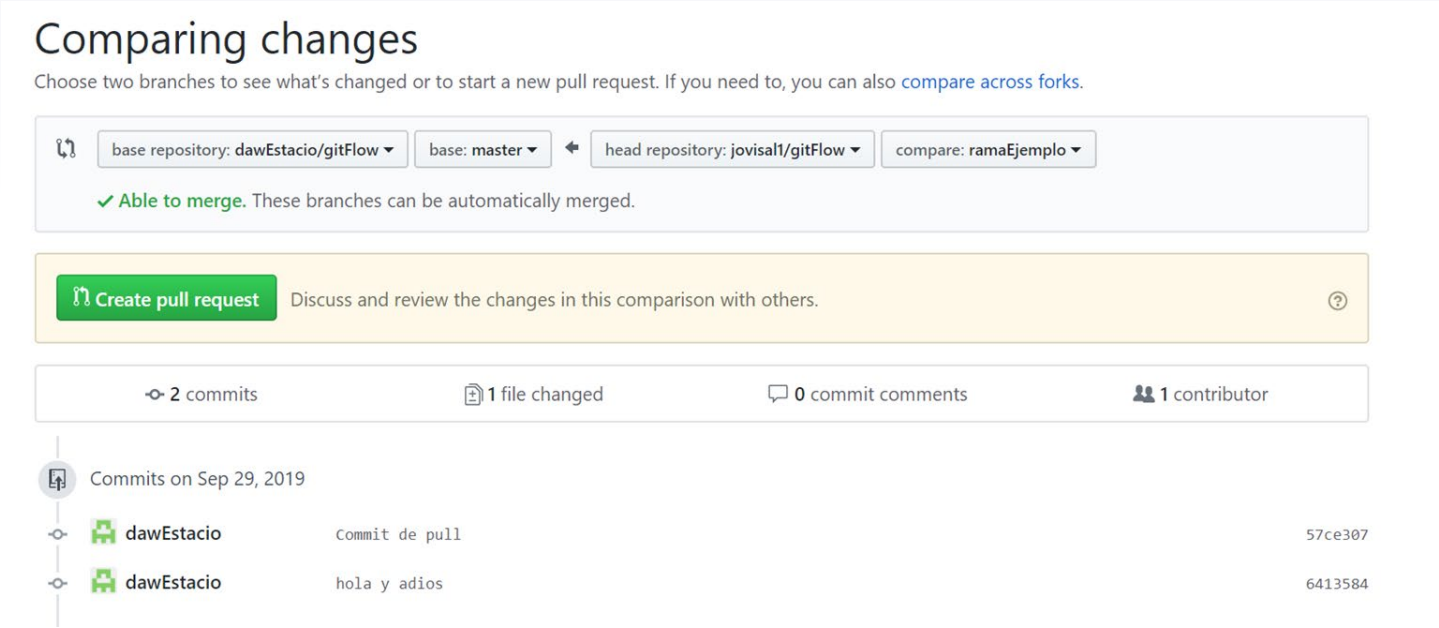
Col·laborar en projectes

Pull request

GitHub ens portarà a una pàgina on podrem comparar els canvis entre les diverses branques i, seleccionant l'opció "compare across forks" també entre els forks del repositori. A continuació, seleccionarem com a "base fork" la branca màster del repositori original, i com a "head fork" la branca que desitgem del nostre fork (en el nostre cas brancaExemple).

Col·laborar en projectes

Pull request



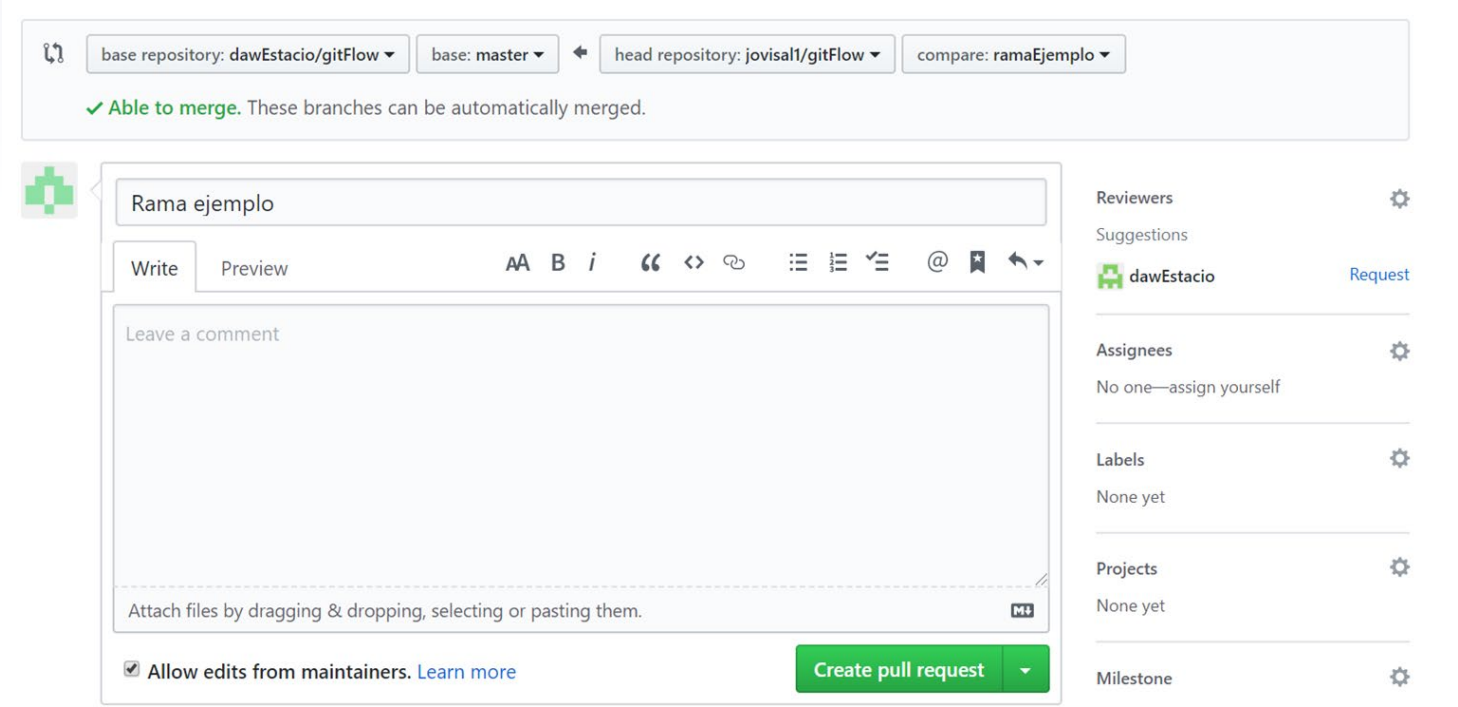
Col·laborar en projectes

Pull request

Després d'apretar "Create pull request", GitHub ens mostrarà una pàgina similar a la que ens mostra en crear un issue. En aquesta pàgina podrem resumir i explicar els canvis realitzats perquè a l'autor del repositori original li resulte més senzill saber quins canvis hem fet i amb quina finalitat. També podrem demanar a usuaris concrets de GitHub que revisen els canvis per donar-los el vistiplau. Normalment aquests usuaris seran les persones encarregades de mantenir el projecte.

Col·laborar en projectes

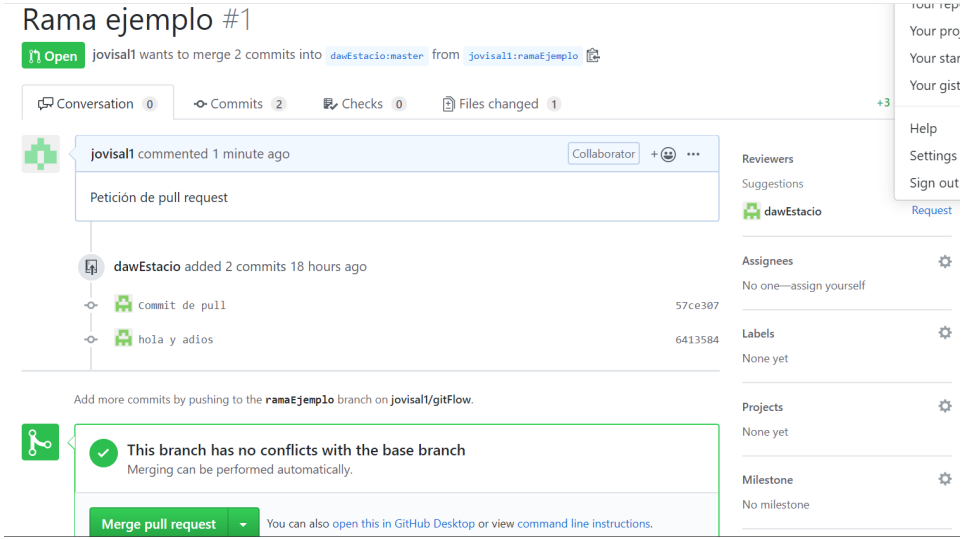
Pull request



Col·laborar en projectes

Pull request

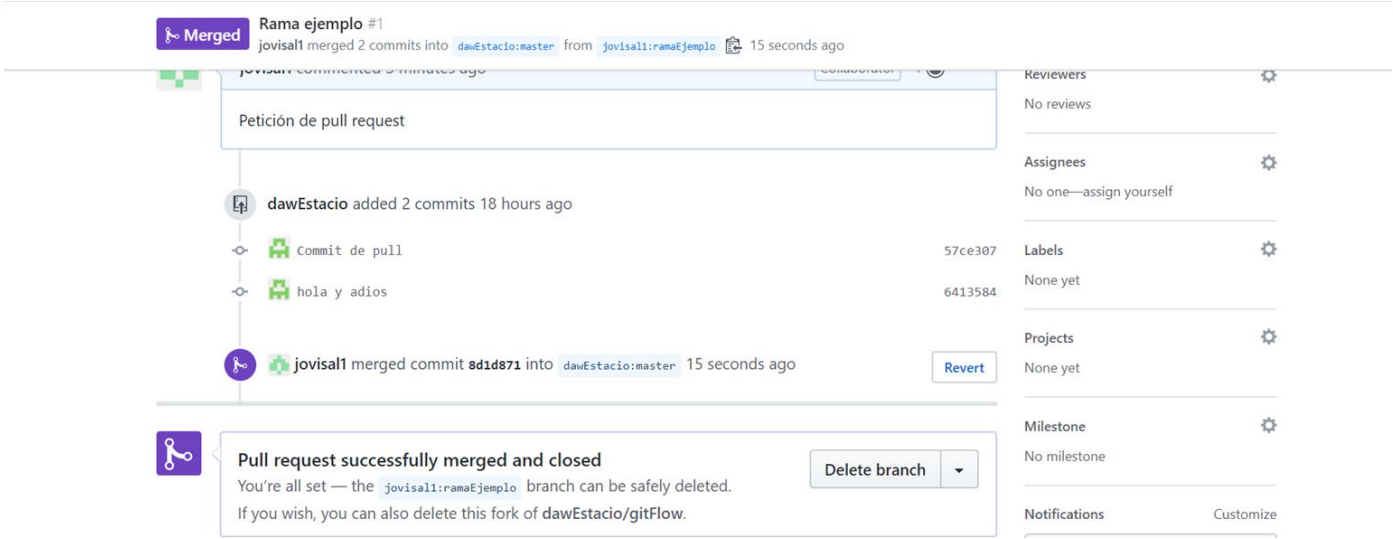
Una vegada emplenat el formulari, fem clic a "Create pull request". Només queda esperar que des del projecte original ens accepten els canvis.



Col·laborar en projectes

Pull request

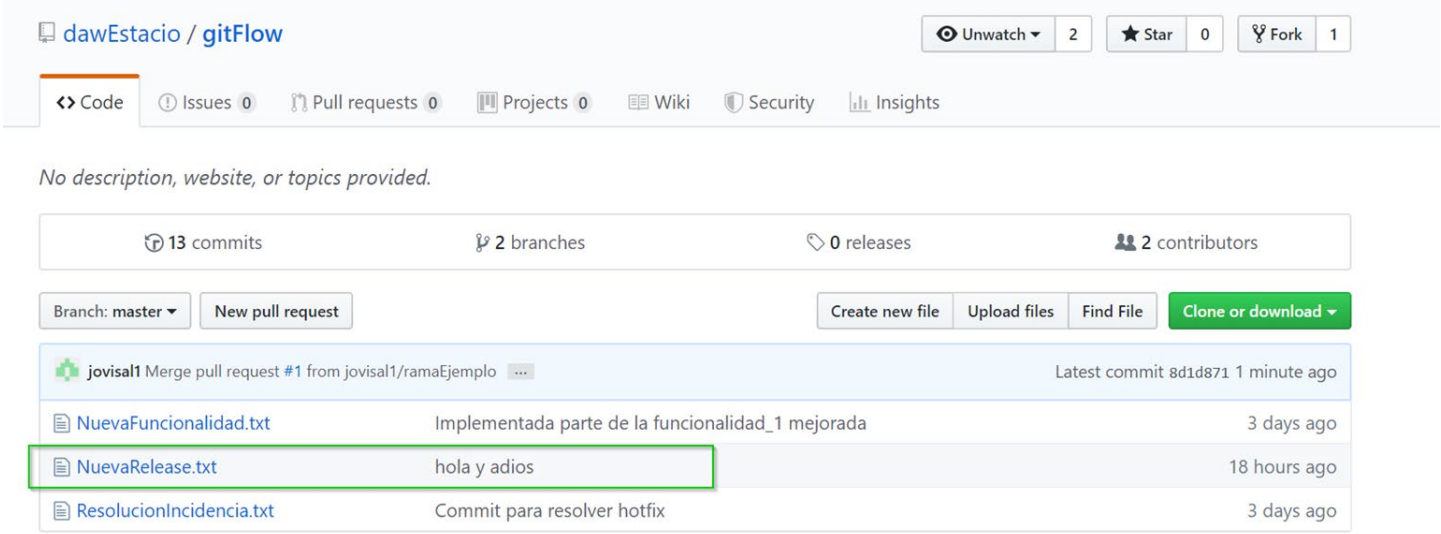
En cas que s'haja dut a terme la integració amb èxit dels nostres canvis, el pull request passarà estar marcat com a "merged"



Col·laborar en projectes

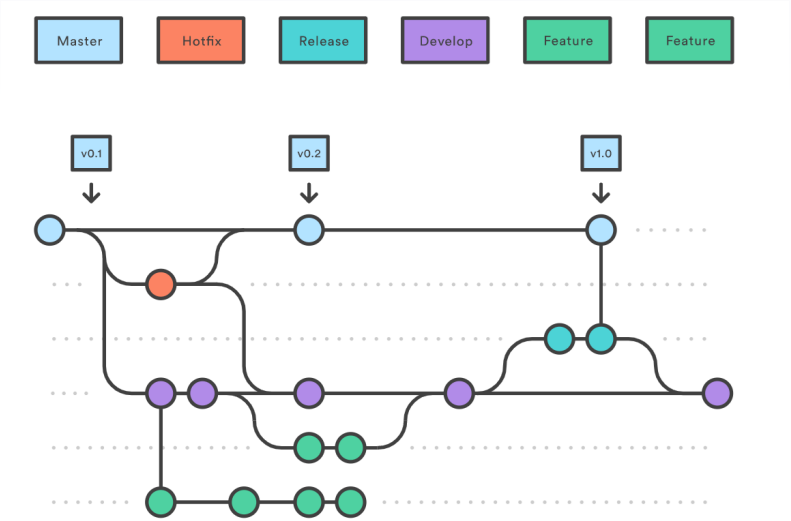
Pull request

Visitant la pàgina principal podrem comprovar que els canvis s'han aplicat amb èxit al repositori original:



Bones pràctiques Git

Seguir una metodologia de treball amb Git acordant les branques necessàries i quan caldrà crear-ne de noves.



Bones pràctiques Git

Missatges de commit

Han de ser simples i clars i és aconsellable tenir en compte:

- Separar l'assumpte del cos del missatge per una línia en blanc.
- L'assumpte no ha de superar els 50 caràcters, deu iniciar-se amb lletra majúscula i no finalitzar amb punt. S'ha d'usar la forma imperativa a la redacció.
- El cos del missatge és aconsellable que tinga línies amb una amplada de no més de 72 caràcters per facilitar-ne la lectura.
- Utilitzeu el cos del missatge per explicar què s'ha resolt i com.

https://chris.beams.io/posts/git-commit/

Bones pràctiques Git

Afegir només els arxius amb què hem treballat

Han de ser simples i clars i és aconsellable tenir en compte:

- Usar *git add .* no és la millor pràctica, el millor en aquests casos és afegir només els arxius on haguem estat treballant, això evita que s'afegeixen molts arxius "trash" que poden portar una repercussió greu en la dimensió del repositori.

- En el seu lloc, és convenient afegir els fitxers individualment (*git add documento.txt*).

Bones pràctiques Git

Evitar commits amb codis incomplets o barrejats

Evitar, en la mesura que siga possible, pujar al repositori remot commits amb canvis de codi que suposen la resolució parcial d'un desenvolupament. Cal compartir codi complet.

D'altra banda, és important fer commits per funcionalitat o requeriment. Això permet definir un abast als commits. Es deu **incloure només fitxers** que estiguen **relacionats amb la funcionalitat** que estiguem desenvolupant, això permet revertir (revert) confirmacions en un futur sense cap problema. Si dins dels **canvis pendents** (pending changes) es troben fitxers **no relacionats** amb la funcionalitat és millor **incloure'ls en un altre commit**

Resum ordres Git

- Creació
  - Clonar un repositori existent `git clone url_repositori`
  - Crear un repositori local `git init`
- Canvis locals
  - Verificar canvis al repositori local `git status`
  - Afegir canvis d'un fitxer determinat `git add nom_fitxer`
  - Commit de canvis `git commit -m "Missatge de commit"`
- Historial de commits
  - Mostra tots els commits començant des del més recent `git log`
  - Mostra els darrers n commits començant des del més recent `git log -n`

Resum ordres Git

- Actualització i publicació
  - Descarregar canvis del repositori remot sense integrar-los al HEAD `git fetch <repo_remoto>`
  - Descarregar canvis del repositori remot i integrar-los/mergejar-los al HEAD `git pull`
  - Publicar canvis al repositori remot `git push origin master`
- Branxes
  - Crear una branca `git branch nom_branca`
  - Llistar branques `git branch -a`
  - Canviar el HEAD de branca `git checkout nom_branca`
  - Crear una branca i posicionar el HEAD en aquesta `git checkout -b nom_branca`
  - Publicació en remot `git push origin nom_branca`
  - Eliminar una branca local `git branch -d nom_branca`

Resum ordres Git

- Branques
  - Merge d'una branca a HEAD `git merge nom_branca`
- Tags
  - Crear una etiqueta `git tag -a nom_etiqueta "Comentari_etiqueta"`
  - Eliminar una etiqueta
    - Eliminem en local `git tag -d nom_tag`
    - Pugem el canvi a remot `git push origin :refs/tags/nom_tag`
  - Publicar etiqueta al servidor remot `git push origin nom_etiqueta`
  - Publicar totes les etiquetes al servidor remot alhora `git push origin --tags`